

TIPOS BÁSICOS

Primitivos

```
string : string — texto
number : number — entero o decimal
boolean : boolean — true/false
bigint : bigint — enteros grandes
symbol : symbol — identificador único
null : null — ausencia intencional
undefined : undefined — sin valor asignado
```

Especiales de TypeScript

```
any : any — desactiva el tipado
unknown : unknown — tipo seguro — verificar antes
never : never — nunca ocurre — fn que lanza
void : void — fn sin retorno explícito
object : object — cualquier no-primitivo
```

Compuestos

```
let arr: number[] = [1,2,3];
let arr: Array<number> = [1,2,3];
let t: [string, number];
// tuple — orden fijo
enum Dir { Up, Down, Left, Right }
// evitar enum — usar union:
type Dir = 'up'|'down'|'left';
```

TYPE vs INTERFACE

interface — para objetos y clases

```
interface User {
  name: string;
  age?: number; // opcional
  readonly id: string;
}
// Extensión:
interface Admin extends User {
  role: string;
}
// Declaración múltiple (merging):
interface User { email: string; }
// se fusionan automáticamente

type — para todo lo demás

type User = {
  name: string;
  age?: number;
};
// Union:
type ID = string | number;
// Intersection:
type Admin = User & { role: string };
// Alias de primitivo:
type Name = string;
// Tuple con nombre:
type Point = [x: number, y: number];
```

¿Cuándo usar cada uno?

interface	objetos, APIs públicas, clases
type	unions, intersections, alias

MODIFICADORES

& tipos especiales

Modificadores de propiedad

```
interface Config {
  readonly host: string;
  // ↑ no se puede reasignar
  port?: number;
  // ↑ opcional — puede faltar
}
```

Union & Intersection

```
// Union — uno u otro:
type Input = string | number;
type Status = 'ok'|'error'|'pending';
// Intersection — ambos a la vez:
type AB = TypeA & TypeB;
```

Literal types

```
type Dir = 'north'|'south';
type Code = 200 | 404 | 500;
// Template literal type:
type Key = `on${string}`;
// onClick, onFocus, onBlur...
```

Inferencia de tipos

```
// TS infiere si no anotas:
let x = 42;
// x es number automáticamente
const fn = () => 'hola';
// retorno inferido: string
// Anotar cuando la inferencia falla:
const arr: string[] = [];
```

Aserciones de tipo

```
// as — forzar tipo (usar con cuidado):
const x = val as string;
// as const — inferir literal:
const cfg = { port: 3000 } as const;
// cfg.port es 3000, no number
// satisfies — validar sin perder tipo:
const c = cfg satisfies Config;
```

ERRORES COMUNES — Tipos, Type vs Interface

```
// TypeScript = JavaScript + Tipos
const arr: any[] = [1,2,3]; // any desactiva el sistema de tipos
const arr: unknown = [1,2,3]; // any acepta todo sin verificar. Usar unknown y hacer type guard.
const arr: { id: string; name: string; } = { id: '1', name: 'Oscar' };
// object vs {} vs Record<string,unknown>
// object: no-primitivo. {}: casi cualquier cosa. Record<K,V>: mapa tipado.
```

```
// TypeScript = JavaScript + Tipos
type Result = { data: T };
interface Result { data: T };
// interface no puede hacer unions ni alias
type Status = 'ok' | 'error'; // interface no puede hacer esto
// null y undefined en strict mode
// Con strictNullChecks, string ≠ string | null. Deben anotarse explícitamente.
```

```
const name: string = 'Oscar';
// anotación de tipo
```

```
type ID = string | number;
// union type
```

```
interface User { name: string; }
// interface
```

```
unknown vs any
// unknown es seguro
```



FUNCIONES

tipadas

Parámetros y retorno

```
// Anotación completa:
function add(
  a: number,
  b: number
): number {
  return a + b;
}
```

Arrow tipada

```
const add =
  (a: number, b: number): number
  => a + b;
```

Parámetros opcionales y por defecto

```
function greet(
  name: string,
  greeting?: string,
  times: number = 1
): string { ... }
```

Rest params

```
function sum(...nums: number[]): number {
  return nums.reduce((a,b) => a+b, 0);
}
```

void vs never

```
// void — no retorna valor útil:
function log(msg: string): void {
  console.log(msg);
}

// never — nunca termina:
function fail(msg: string): never {
  throw new Error(msg);
}
```

Overloads

```
function fmt(x: string): string;
function fmt(x: number): string;
function fmt(x: any): string {
  return String(x);
}
```



GENERICS

Función genérica

```
// T es el parámetro de tipo
function identity<T>(value: T): T {
  return value;
}

identity<string>('hola');
identity(42); // T inferido
```

Múltiples generics

```
function pair<A, B>(
  a: A, b: B
): [A, B] {
  return [a, b];
}
```

Constraints — extends

```
// T debe tener .length
function longest<T extends { length: number }>(
  a: T, b: T
): T {
  return a.length >= b.length ? a : b;
}
```

Generic en interface

```
interface Repository<T> {
  findById(id: string): T;
  findALL(): T[];
  save(item: T): void;
}
```

Generic en clase

```
class Stack<T> {
  private items: T[] = [];
  push(item: T): void {
    this.items.push(item);
  }
  pop(): T | undefined {
    return this.items.pop();
  }
}
```

keyof y typeof

```
type Keys = keyof User;
// 'name' | 'age' | 'id'

type UserType = typeof userObj;
// infiere el tipo del valor

function get<T, K extends keyof T>(
  obj: T, key: K
): T[K] { return obj[key]; }
```



UTILITY TYPES

& narrowing

Utility Types — los más usados

Partial<T> todos los campos opcionales

Required<T> todos los campos obligatorios

Readonly<T> todos los campos readonly

Pick<T, K> solo las keys K de T

Omit<T, K> T sin las keys K

Record<K, V> objeto con keys K y valores V

ReturnType<T> tipo de retorno de una fn

Parameters<T> tipos de los parámetros

Awaited<T> desenvuelve Promise<T>

NonNullable<T> excluye null y undefined

Exclude<T, U> quita U del union T

Extract<T, U> solo lo común entre T y U

Narrowing — acotar el tipo

```
// typeof guard:
if (typeof x === 'string') {
  x.toUpperCase(); // string seguro
}

// instanceof guard:
if (err instanceof Error) {
  err.message; // Error seguro
}

// in guard:
if ('name' in obj) {
  obj.name; // existe la propiedad
}

// Type predicate:
function isStr(x: unknown): x is string {
  return typeof x === 'string';
}
```

ERRORES COMUNES — Funciones, Generics y Utility Types

```
// TypeScript = JavaScript + Tipos
const sum = (a: number, b: number): number => findALL(): Promise<T[]>;
// Error: valuse as string le dice a TS que confíe en ti. Si te equivocas, error en runtime.

Generic sin constraint acepta cualquier cosa
function fn<T>(x: T) — T puede ser null, undefined, object... Usar extends.
```

```
type Result<T> =
  true | data: T;

Olvidar anotar el tipo de retorno
TS infiere, pero anotar explícitamente documenta la intención y evita sorpresas.

Partial no hace deep optional
Partial<{ a: { b: string } }> hace a opcional pero b sigue siendo required.
```

function id<T>(x: T): T {

return x; }

Partial<User>

// todos opcionales

type Guard = x is string

// type predicate

ReturnType<typeof fn>

// inferir retorno



CLASES

en TypeScript

Modificadores de acceso

```
class BankAccount {
  public owner: string;
  // ↑ accesible desde cualquier lugar
  private balance: number;
  // ↑ solo dentro de la clase
  protected id: string;
  // ↑ clase y subclases
  readonly createdAt: Date;
  // ↑ solo lectura
}
```

Parameter properties — shorthand

```
// Forma larga:
class User {
  public name: string;
  constructor(name: string) {
    this.name = name;
  }
}
// Forma corta (equivalente):
class User {
  constructor(public name: string) {}
}
```

implements vs extends

```
interface Printable {
  print(): void;
}
// implements — cumplir contrato:
class Doc implements Printable {
  print() { console.log(this); }
}
// extends — heredar:
class Admin extends User {
  constructor(name: string,
    public role: string) {
    super(name);
  }
}
```

Abstract classes

```
abstract class Shape {
  abstract area(): number;
  describe() {
    return `Area: ${this.area()}`;
  }
}
class Circle extends Shape {
  constructor(private r: number) { super(); }
  area() { return Math.PI * this.r ** 2; }
}
```



TSCONFIG

opciones esenciales

tsconfig.json — opciones esenciales

Verificación de tipos

```
"strict": true — activa todas las opciones strict
"strictNullChecks" — null/undefined deben anotarse
"noImplicitAny" — prohíbe any implícito
"noUncheckedIndexedAccess" — arr[i] puede ser undefined
```

Compilación

```
"target": "ES2022" — versión JS de salida
"module": "NodeNext" — sistema de módulos
"lib": ["ES2022", "DOM"] — APIs disponibles
"outDir": "./dist" — carpeta de salida
"rootDir": "./src" — carpeta de entrada
"declaration": true — genera archivos .d.ts
"sourceMap": true — mapas para debugging
```

Rutas y módulos

```
"baseUrl": "." — base para rutas absolutas
"paths": {"@/*": ["./src/*"]} — alias de importación
"moduleResolution": "Bundler" — para Vite/webpack
"esModuleInterop": true — compatibilidad CommonJS
```

Incluir / excluir

```
"include": ["src/**/*.ts"] — archivos a compilar
"exclude": ["node_modules", "dist"] — archivos a ignorar
```

Configuración recomendada para Next.js 15:

```
{ "extends": "next/core-web-vitals",
  "compilerOptions": {
    "strict": true,
    "target": "ES2022" } }
```



PATRONES

avanzados

as const — inferir literal

```
const config = {
  port: 3000,
  host: 'localhost'
} as const;
// port es 3000, no number
// host es 'localhost', no string
```

satisfies — validar sin perder tipo

```
const palette = {
  red: [255, 0, 0],
  blue: '#0000ff'
} satisfies Record<string, string | number[]>;
// TS valida Y mantiene los tipos literales
```

Discriminated unions

```
type Shape =
  | { kind: 'circle'; r: number }
  | { kind: 'rect'; w: number; h: number };
function area(s: Shape): number {
  switch (s.kind) {
    case 'circle': return Math.PI * s.r ** 2;
    case 'rect': return s.w * s.h;
  }
}
```

infer — extraer tipos

```
type Unwrap<T> =
  T extends Promise<infer U>
    ? U : T;
// Unwrap<Promise<string>> := string
```

Enums vs Union literals

```
// Evitar enum en la mayoría de casos:
enum Dir { Up, Down } // ▲
// Preferir union literal:
type Dir = 'up' | 'down'; // ◻
// const enum si necesitas enum:
const enum Dir { Up = 'UP' } // ◻
```

Declaration merging

```
// Extender tipos de librerías:
declare module 'express' {
  interface Request {
    user?: User;
  }
}
```

ERRORES COMUNES — Clases, tsconfig y Patrones

```
// TypeScript = JavaScript + Tipos
const suma = (a: number, b: number): Promise<T> => {
  // "strict: false" en producción
  // "strict: false" desactiva strictNullChecks, noImplicitAny y más. Siempre true.
  return Promise.resolve(a + b);
}
enum en lugar de union literal
enum genera código JS extra. 'type Dir = "up"|"down"' es más simple y seguro.
```

```
type Result<T> =
  { ok: true; data: T }
  | { ok: false; error: string };
paths sin baseUrl en tsconfig
"paths" requiere "baseUrl". Sin él los alias no funcionan.
private de TS vs #private de JS
private de TS solo existe en compilación. #campo es realmente privado en runtime.
```

```
class A { #x: number = 0; }
// campo privado
```

```
"strict": true
// siempre en prod.
```

```
as const satisfies Config
// ES2024+
```

```
keyof T
// claves del tipo
```